

Terraform Multi-Cloud Lab Manual

Hands-on Labs for AWS, Azure, GCP, and Multi-Cloud — Basic to Advanced

Infrastructure as Code Training Series

oday

Contents

1	How to Use This Manual	4
1.1	Prerequisites (all labs)	4
1.2	Standard Lab Format	4
1.3	General Terraform Workflow (applies to every lab)	4
1.4	Safety & Cost Notes	4
2	Part 1: AWS Labs	6
2.1	Lab AWS-1: Deploy Your First EC2 Instance (Basic)	6
2.1.1	Objective	6
2.1.2	Prerequisites	6
2.1.3	Architecture Overview	6
2.1.4	Step-by-Step Instructions	6
2.1.5	Cleanup	8
2.1.6	Challenge / Extension	8
2.1.7	Troubleshooting Notes	8
2.2	Lab AWS-2: VPC with Public/Private Subnets and a Web Server (Basic-Moderate)	9
2.2.1	Objective	9
2.2.2	Prerequisites	9
2.2.3	Architecture Overview	9
2.2.4	Step-by-Step Instructions	9
2.2.5	Cleanup	12
2.2.6	Challenge / Extension	12
2.2.7	Troubleshooting Notes	12
2.3	Lab AWS-3: Static Website on S3 with Remote State (Moderate)	13
2.3.1	Objective	13
2.3.2	Prerequisites	13
2.3.3	Architecture Overview	13
2.3.4	Step-by-Step Instructions	13
2.3.5	Cleanup	16
2.3.6	Challenge / Extension	16
2.3.7	Troubleshooting Notes	16
2.4	Lab AWS-4: Highly Available Web App with ALB, Auto Scaling, and RDS (Advanced)	17
2.4.1	Objective	17
2.4.2	Prerequisites	17
2.4.3	Architecture Overview	17
2.4.4	Step-by-Step Instructions	17
2.4.5	Cleanup	23
2.4.6	Challenge / Extension	23
2.4.7	Troubleshooting Notes	23

3	Part 2: Azure Labs	24
3.1	Lab AZ-1: Deploy Your First Linux VM on Azure (Basic)	24
3.1.1	Objective	24
3.1.2	Prerequisites	24
3.1.3	Architecture Overview	24
3.1.4	Step-by-Step Instructions	24
3.1.5	Cleanup	27
3.1.6	Challenge / Extension	27
3.1.7	Troubleshooting Notes	27
3.2	Lab AZ-2: Multi-Tier VNet with NSG Rules and Availability Set (Basic-Moderate)	28
3.2.1	Objective	28
3.2.2	Prerequisites	28
3.2.3	Architecture Overview	28
3.2.4	Step-by-Step Instructions	28
3.2.5	Cleanup	31
3.2.6	Challenge / Extension	31
3.2.7	Troubleshooting Notes	32
3.3	Lab AZ-3: Remote State in Azure Storage + App Service with SQL Database (Moderate)	33
3.3.1	Objective	33
3.3.2	Prerequisites	33
3.3.3	Architecture Overview	33
3.3.4	Step-by-Step Instructions	33
3.3.5	Cleanup	36
3.3.6	Challenge / Extension	36
3.3.7	Troubleshooting Notes	36
3.4	Lab AZ-4: AKS (Azure Kubernetes Service) Cluster with Terraform (Advanced)	37
3.4.1	Objective	37
3.4.2	Prerequisites	37
3.4.3	Architecture Overview	37
3.4.4	Step-by-Step Instructions	37
3.4.5	Cleanup	39
3.4.6	Challenge / Extension	39
3.4.7	Troubleshooting Notes	39
4	Part 3: GCP Labs	40
4.1	Lab GCP-1: Deploy Your First Compute Engine VM (Basic)	40
4.1.1	Objective	40
4.1.2	Prerequisites	40
4.1.3	Architecture Overview	40
4.1.4	Step-by-Step Instructions	40
4.1.5	Cleanup	41
4.1.6	Challenge / Extension	42
4.1.7	Troubleshooting Notes	42
4.2	Lab GCP-2: Custom VPC with Subnet, Firewall Rules, and Multiple Instances (Basic-Moderate)	43
4.2.1	Objective	43
4.2.2	Prerequisites	43
4.2.3	Architecture Overview	43
4.2.4	Step-by-Step Instructions	43
4.2.5	Cleanup	45
4.2.6	Challenge / Extension	45
4.2.7	Troubleshooting Notes	45
4.3	Lab GCP-3: Remote State in GCS + Cloud SQL Backend (Moderate)	46
4.3.1	Objective	46
4.3.2	Prerequisites	46
4.3.3	Architecture Overview	46

4.3.4	Step-by-Step Instructions	46
4.3.5	Cleanup	49
4.3.6	Challenge / Extension	49
4.3.7	Troubleshooting Notes	49
4.4	Lab GCP-4: GKE (Google Kubernetes Engine) Cluster with Modules (Advanced)	50
4.4.1	Objective	50
4.4.2	Prerequisites	50
4.4.3	Architecture Overview	50
4.4.4	Step-by-Step Instructions	50
4.4.5	Cleanup	52
4.4.6	Challenge / Extension	52
4.4.7	Troubleshooting Notes	53
5	Part 4: Multi-Cloud Labs	54
5.1	Lab MC-1: Provision Storage Simultaneously on AWS, Azure, and GCP (Basic)	54
5.1.1	Objective	54
5.1.2	Prerequisites	54
5.1.3	Architecture Overview	54
5.1.4	Step-by-Step Instructions	54
5.1.5	Cleanup	56
5.1.6	Challenge / Extension	56
5.1.7	Troubleshooting Notes	56
5.2	Lab MC-2: Consistent VM Deployment Across AWS, Azure, and GCP Using a Shared Variable Pattern (Moderate)	57
5.2.1	Objective	57
5.2.2	Prerequisites	57
5.2.3	Architecture Overview	57
5.2.4	Step-by-Step Instructions	57
5.2.5	Cleanup	62
5.2.6	Challenge / Extension	62
5.2.7	Troubleshooting Notes	62
5.3	Lab MC-3: Multi-Cloud Networking with Planned CIDR Ranges and Cross-Cloud Outputs (Moderate-Advanced)	63
5.3.1	Objective	63
5.3.2	Prerequisites	63
5.3.3	Architecture Overview	63
5.3.4	Step-by-Step Instructions	63
5.3.5	Cleanup	65
5.3.6	Challenge / Extension	65
5.3.7	Troubleshooting Notes	66
5.4	Lab MC-4: Multi-Cloud Primary/Standby Pattern with Workspaces and Shared Modules (Advanced)	67
5.4.1	Objective	67
5.4.2	Prerequisites	67
5.4.3	Architecture Overview	67
5.4.4	Step-by-Step Instructions	67
5.4.5	Cleanup	73
5.4.6	Challenge / Extension	73
5.4.7	Troubleshooting Notes	73
6	Appendix: Full Cleanup Checklist	74

1 How to Use This Manual

This manual contains **16 hands-on Terraform labs**: 4 each for AWS, Azure, and GCP, plus 4 multi-cloud labs that combine two or more providers in a single workflow. Each track progresses **Basic** → **Basic/Moderate** → **Moderate** → **Advanced**.

1.1 Prerequisites (all labs)

- A computer with internet access (Linux, macOS, or Windows with WSL recommended)
- Terraform CLI installed (version 1.6 or later)
 - Verify with: `terraform -version`
 - If not installed, follow: <https://developer.hashicorp.com/terraform/install>
- A text editor (VS Code recommended, with the HashiCorp Terraform extension)
- Basic comfort with a terminal/command line
- An active account on the relevant cloud provider(s):
 - **AWS**: an AWS account with an IAM user that has programmatic (CLI) access
 - **Azure**: an Azure subscription (Free Tier is fine)
 - **GCP**: a Google Cloud project with billing enabled (Free Tier / trial credits are fine)
- A credit card on file with the provider (required for account creation) — labs are designed to use free-tier eligible resources wherever possible, but **students must destroy resources after each lab** to avoid charges

1.2 Standard Lab Format

Every lab in this manual follows the same structure so students always know what to expect:

1. **Objective** — what you will build and learn
2. **Difficulty & Estimated Time**
3. **Prerequisites** — anything specific to that lab
4. **Architecture Overview** — a short description of what gets created
5. **Step-by-Step Instructions** — numbered, copy-pasteable steps
6. **Validation** — how to confirm it worked
7. **Cleanup** — how to tear everything down (very important — always run this!)
8. **Challenge / Extension** — optional stretch goal for fast finishers
9. **Troubleshooting Notes**

1.3 General Terraform Workflow (applies to every lab)

Students should internalize this core loop, used in every single lab:

```
terraform init      # Downloads providers/modules, sets up backend
terraform fmt       # Auto-formats code (good habit before every commit)
terraform validate  # Checks syntax and internal consistency
terraform plan      # Shows what WILL change (dry run) - always read this!
terraform apply     # Creates/updates real infrastructure (type 'yes' to confirm)
terraform show      # Inspect current state
terraform destroy   # Tears down everything Terraform created (type 'yes' to confirm)
```

Golden rule for students: never skip `terraform plan` before `apply`, and never forget `terraform destroy` at the end of a lab.

1.4 Safety & Cost Notes

- Every lab uses small/free-tier-eligible instance sizes (e.g., `t2.micro`/`t3.micro` on AWS, `Standard_B1s` on Azure, `e2-micro` on GCP). Instructors can substitute larger sizes if the classroom has larger budgets.

- Labs involving managed Kubernetes (EKS/AKS/GKE) and managed databases (RDS/Azure SQL/Cloud SQL) **do incur cost** even at the smallest sizes — this is called out explicitly in those labs, along with expected approximate run cost per hour.
- Always run **terraform destroy** at the end of a lab, and double check in the cloud console that no resources remain.

2 Part 1: AWS Labs

2.1 Lab AWS-1: Deploy Your First EC2 Instance (Basic)

Difficulty: Basic | **Estimated Time:** 30–40 minutes

2.1.1 Objective

Learn the core Terraform workflow by provisioning a single Amazon EC2 instance and a security group using the AWS provider.

2.1.2 Prerequisites

- AWS account with an IAM user that has `AmazonEC2FullAccess` (or equivalent) permissions
- AWS Access Key ID and Secret Access Key
- AWS CLI installed and configured (`aws configure`), OR environment variables set for credentials

2.1.3 Architecture Overview

You will create: - One security group allowing inbound SSH (port 22) and HTTP (port 80) - One `t2.micro` EC2 instance running Amazon Linux 2, running a simple Apache web server via user data

2.1.4 Step-by-Step Instructions

Step 1 — Set up your project folder

```
mkdir terraform-aws-lab1
cd terraform-aws-lab1
```

Step 2 — Configure AWS credentials

```
aws configure
# AWS Access Key ID [None]: <your-access-key>
# AWS Secret Access Key [None]: <your-secret-key>
# Default region name [None]: us-east-1
# Default output format [None]: json
```

Step 3 — Create the provider configuration

Create a file named `provider.tf`:

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Step 4 — Find a valid AMI using a data source

Create `main.tf`:

```
data "aws_ami" "amazon_linux" {
  most_recent = true
}
```

```

owners      = ["amazon"]

filter {
  name      = "name"
  values    = ["amzn2-ami-hvm-*-x86_64-gp2"]
}
}

resource "aws_security_group" "web_sg" {
  name          = "lab1-web-sg"
  description   = "Allow SSH and HTTP"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "lab1-web-sg"
  }
}

resource "aws_instance" "web" {
  ami              = data.aws_ami.amazon_linux.id
  instance_type    = "t2.micro"
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    yum update -y
    yum install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo "<h1>Hello from Terraform Lab 1</h1>" > /var/www/html/index.html
  EOF
}

```

```
tags = {
  Name = "lab1-web-instance"
}

output "instance_public_ip" {
  value = aws_instance.web.public_ip
}
```

Step 5 — Initialize Terraform

```
terraform init
```

You should see: Terraform has been successfully initialized!

Step 6 — Format and validate

```
terraform fmt
terraform validate
```

Step 7 — Plan

```
terraform plan
```

Read the plan carefully. It should show 2 to add (security group + instance).

Step 8 — Apply

```
terraform apply
```

Type **yes** when prompted. Wait for completion (~1–2 minutes).

Step 9 — Validate

```
terraform output instance_public_ip
```

Copy the IP address and open `http://<that-ip>` in a browser. You should see “Hello from Terraform Lab 1”.

2.1.5 Cleanup

```
terraform destroy
```

Type **yes** when prompted. Confirm in the AWS EC2 console that the instance is terminated.

2.1.6 Challenge / Extension

- Add a variable for `instance_type` so it can be changed without editing code.
- Add an `aws_key_pair` resource and associate it so you can actually SSH in.

2.1.7 Troubleshooting Notes

- **UnauthorizedOperation:** your IAM user lacks EC2 permissions — attach **AmazonEC2FullAccess**.
- **InvalidAMIID.NotFound:** the AMI is region-specific; make sure your **provider** region matches where the AMI filter runs.
- If `terraform destroy` hangs, check the AWS console — some resources (like ENIs) can have brief propagation delays.

2.2 Lab AWS-2: VPC with Public/Private Subnets and a Web Server (Basic–Moderate)

Difficulty: Basic–Moderate | **Estimated Time:** 45–60 minutes

2.2.1 Objective

Build a custom VPC with public and private subnets, an Internet Gateway, route tables, and an EC2 instance in the public subnet — the foundational networking pattern used in almost all real AWS deployments.

2.2.2 Prerequisites

- Completion of Lab AWS-1 (or equivalent familiarity with the Terraform workflow)
- Same AWS credentials setup

2.2.3 Architecture Overview

- 1 VPC (10.0.0.0/16)
- 1 public subnet (10.0.1.0/24) + 1 private subnet (10.0.2.0/24)
- 1 Internet Gateway attached to the VPC
- 1 route table for the public subnet routing 0.0.0.0/0 to the IGW
- 1 EC2 instance in the public subnet with a security group

2.2.4 Step-by-Step Instructions

Step 1 — Project setup

```
mkdir terraform-aws-lab2 && cd terraform-aws-lab2
```

Step 2 — provider.tf (same as Lab AWS-1)

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Step 3 — variables.tf

```
variable "vpc_cidr" {
  default = "10.0.0.0/16"
}

variable "public_subnet_cidr" {
  default = "10.0.1.0/24"
}

variable "private_subnet_cidr" {
  default = "10.0.2.0/24"
}
```

```
variable "availability_zone" {  
  default = "us-east-1a"  
}
```

Step 4 — network.tf

```
resource "aws_vpc" "main" {  
  cidr_block      = var.vpc_cidr  
  enable_dns_support = true  
  enable_dns_hostnames = true  
  tags = { Name = "lab2-vpc" }  
}  
  
resource "aws_internet_gateway" "igw" {  
  vpc_id = aws_vpc.main.id  
  tags = { Name = "lab2-igw" }  
}  
  
resource "aws_subnet" "public" {  
  vpc_id            = aws_vpc.main.id  
  cidr_block        = var.public_subnet_cidr  
  availability_zone  = var.availability_zone  
  map_public_ip_on_launch = true  
  tags = { Name = "lab2-public-subnet" }  
}  
  
resource "aws_subnet" "private" {  
  vpc_id            = aws_vpc.main.id  
  cidr_block        = var.private_subnet_cidr  
  availability_zone  = var.availability_zone  
  tags = { Name = "lab2-private-subnet" }  
}  
  
resource "aws_route_table" "public_rt" {  
  vpc_id = aws_vpc.main.id  
  
  route {  
    cidr_block = "0.0.0.0/0"  
    gateway_id = aws_internet_gateway.igw.id  
  }  
  
  tags = { Name = "lab2-public-rt" }  
}  
  
resource "aws_route_table_association" "public_assoc" {  
  subnet_id      = aws_subnet.public.id  
  route_table_id = aws_route_table.public_rt.id  
}
```

Step 5 — compute.tf

```
data "aws_ami" "amazon_linux" {  
  most_recent = true  
  owners      = ["amazon"]  
  filter {  
    name = "name"
```

```

    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

resource "aws_security_group" "web_sg" {
  name     = "lab2-web-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "lab2-web-sg" }
}

resource "aws_instance" "web" {
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = "t2.micro"
  subnet_id          = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    yum update -y
    yum install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo "<h1>Lab 2: Custom VPC Web Server</h1>" > /var/www/html/index.html
  EOF

  tags = { Name = "lab2-web-instance" }
}

```

Step 6 — outputs.tf

```

output "vpc_id" {
  value = aws_vpc.main.id
}

output "public_subnet_id" {

```

```
    value = aws_subnet.public.id
  }

  output "web_instance_public_ip" {
    value = aws_instance.web.public_ip
  }
```

Step 7 — Run the workflow

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

Type **yes**. Wait for the ~9 resources to be created.

Step 8 — Validate

```
terraform output web_instance_public_ip
```

Browse to <http://<ip>> — you should see the Lab 2 page. In the AWS Console, confirm the VPC, subnets, and route tables exist and match the diagram.

2.2.5 Cleanup

```
terraform destroy
```

2.2.6 Challenge / Extension

- Add a second private subnet in a different AZ for high availability.
- Add a NAT Gateway so instances in the private subnet can reach the internet outbound-only.

2.2.7 Troubleshooting Notes

- **DependencyViolation on destroy:** usually caused by leftover ENIs from the instance; wait 30 seconds and re-run `terraform destroy`.
- If the instance has no public IP, verify `map_public_ip_on_launch = true` is on the subnet and the instance was launched *after* that subnet setting was applied.

2.3 Lab AWS-3: Static Website on S3 with Remote State (Moderate)

Difficulty: Moderate | **Estimated Time:** 60–75 minutes

2.3.1 Objective

Learn to configure a remote backend (S3 + DynamoDB for state locking) and deploy a static website hosted on S3, fronted by proper bucket policies. This lab introduces **team-safe Terraform**, not just single-user usage.

2.3.2 Prerequisites

- Completion of Labs AWS-1 and AWS-2
- IAM permissions for S3 and DynamoDB

2.3.3 Architecture Overview

- 1 S3 bucket for Terraform remote state + 1 DynamoDB table for state locking (created in a one-time bootstrap step)
- 1 S3 bucket configured for static website hosting, holding `index.html` and `error.html`
- A public-read bucket policy scoped only to that bucket

2.3.4 Step-by-Step Instructions

Step 1 — Bootstrap the backend (one-time, separate mini-project)

```
mkdir terraform-aws-lab3-bootstrap && cd terraform-aws-lab3-bootstrap
```

Create `main.tf`:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "tf_state" {
  bucket = "tf-state-lab3-${random_id.suffix.hex}"
}

resource "aws_s3_bucket_versioning" "tf_state_versioning" {
  bucket = aws_s3_bucket.tf_state.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_dynamodb_table" "tf_lock" {
  name           = "tf-lock-lab3"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key      = "LockID"
}
```

```

    attribute {
      name = "LockID"
      type = "S"
    }
  }

resource "random_id" "suffix" {
  byte_length = 4
}

output "bucket_name" {
  value = aws_s3_bucket.tf_state.bucket
}

```

Run:

```

terraform init
terraform apply

```

Note the `bucket_name` output — you'll need it in Step 2. **Do not destroy this bootstrap stack until the whole lab is complete.**

Step 2 — Main project with remote backend

```

cd ..
mkdir terraform-aws-lab3 && cd terraform-aws-lab3

```

Create `backend.tf` (replace the bucket name with your actual output value):

```

terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket      = "tf-state-lab3-XXXXXXX"
  key         = "lab3/terraform.tfstate"
  region      = "us-east-1"
  dynamodb_table = "tf-lock-lab3"
  encrypt     = true
}

provider "aws" {
  region = "us-east-1"
}

```

Step 3 — Website bucket resources — `main.tf`

```

resource "aws_s3_bucket" "website" {
  bucket = "lab3-static-site-${random_id.site_suffix.hex}"
}

resource "random_id" "site_suffix" {

```

```

    byte_length = 4
  }

resource "aws_s3_bucket_website_configuration" "website_config" {
  bucket = aws_s3_bucket.website.id

  index_document {
    suffix = "index.html"
  }
  error_document {
    key = "error.html"
  }
}

resource "aws_s3_bucket_public_access_block" "public_access" {
  bucket                = aws_s3_bucket.website.id
  block_public_acls     = false
  block_public_policy   = false
  ignore_public_acls   = false
  restrict_public_buckets = false
}

resource "aws_s3_bucket_policy" "public_read" {
  bucket = aws_s3_bucket.website.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid      = "PublicReadGetObject"
      Effect   = "Allow"
      Principal = "*"
      Action   = "s3:GetObject"
      Resource = "${aws_s3_bucket.website.arn}/*"
    }]
  })
  depends_on = [aws_s3_bucket_public_access_block.public_access]
}

resource "aws_s3_object" "index" {
  bucket      = aws_s3_bucket.website.id
  key         = "index.html"
  content     = "<h1>Lab 3: Static Site via Terraform Remote State</h1>"
  content_type = "text/html"
}

resource "aws_s3_object" "error" {
  bucket      = aws_s3_bucket.website.id
  key         = "error.html"
  content     = "<h1>404 - Not Found</h1>"
  content_type = "text/html"
}

output "website_endpoint" {
  value = aws_s3_bucket_website_configuration.website_config.website_endpoint
}

```

Step 4 — Init with remote backend

```
terraform init
```

Terraform confirms it is using the S3 backend. Notice: `terraform.tfstate` is **not** created locally — it now lives in S3, and locking is handled via DynamoDB.

Step 5 — Plan and apply

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

Step 6 — Validate

```
terraform output website_endpoint
```

Open the endpoint URL in a browser (`http://<endpoint>`, note: `http`, not `https`, for S3 website endpoints). Confirm the page loads. Also open the AWS S3 console and confirm the state file exists inside the bootstrap bucket under `lab3/terraform.tfstate`.

Step 7 — Demonstrate locking (optional but recommended for class demo) Open two terminals in the same folder. Run `terraform apply` in both nearly simultaneously — the second one should report **Error** acquiring the state lock, demonstrating DynamoDB-based locking.

2.3.5 Cleanup

```
terraform destroy          # destroys the website stack
cd ../terraform-aws-lab3-bootstrap
terraform destroy          # destroys the state bucket + lock table LAST
```

2.3.6 Challenge / Extension

- Add CloudFront in front of the S3 bucket for HTTPS support.
- Parametrize the bucket name/region using `terraform.tfvars` and a `variables.tf`.

2.3.7 Troubleshooting Notes

- **BucketAlreadyExists:** S3 bucket names are globally unique — the `random_id` suffix helps, but if you rerun the bootstrap you'll get a new name each time; keep track of it.
- If `terraform init` complains about backend changes, run `terraform init -reconfigure`.
- Always destroy the *website* stack before the *bootstrap* stack, since the website's state lives inside the bootstrap bucket.

2.4 Lab AWS-4: Highly Available Web App with ALB, Auto Scaling, and RDS (Advanced)

Difficulty: Advanced | **Estimated Time:** 90–120 minutes **Cost note:** this lab provisions an RDS instance (db.t3.micro) and multiple EC2 instances — expect a small hourly cost (a few cents/hour on free tier eligible sizes). Destroy promptly after the lab.

2.4.1 Objective

Combine everything learned so far into a production-style, highly available 2-tier architecture: a multi-AZ VPC, an Application Load Balancer, an Auto Scaling Group of web servers across two AZs, and an RDS MySQL database in private subnets — all built using reusable **Terraform modules**.

2.4.2 Prerequisites

- Completion of Labs AWS-1 through AWS-3
- Comfortable with basic Terraform module syntax

2.4.3 Architecture Overview

- VPC with 2 public subnets + 2 private subnets across 2 AZs
- Internet Gateway + NAT Gateway (for private subnet outbound access)
- Application Load Balancer (public) → target group → Auto Scaling Group (2–4 EC2 instances across the 2 public subnets, running a simple app)
- RDS MySQL instance in the private subnets (via a DB subnet group), reachable only from the web tier's security group
- Structured using a local module for networking

2.4.4 Step-by-Step Instructions

Step 1 — Project structure

```
mkdir -p terraform-aws-lab4/modules/network
cd terraform-aws-lab4
```

Folder layout:

```
terraform-aws-lab4/
|-- main.tf
|-- variables.tf
|-- outputs.tf
|-- provider.tf
`-- modules/
    |-- network/
        |-- main.tf
        |-- variables.tf
        |-- outputs.tf
```

Step 2 — provider.tf (root)

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

```
provider "aws" {
  region = "us-east-1"
}
```

Step 3 — Network module — modules/network/variables.tf

```
variable "vpc_cidr"          { default = "10.0.0.0/16" }
variable "public_subnet_cidrs" { default = ["10.0.1.0/24", "10.0.2.0/24"] }
variable "private_subnet_cidrs" { default = ["10.0.11.0/24", "10.0.12.0/24"] }
variable "azs"               { default = ["us-east-1a", "us-east-1b"] }
```

Step 4 — modules/network/main.tf

```
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "lab4-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
}

resource "aws_subnet" "public" {
  count            = length(var.public_subnet_cidrs)
  vpc_id          = aws_vpc.main.id
  cidr_block      = var.public_subnet_cidrs[count.index]
  availability_zone = var.azs[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "lab4-public-${count.index}" }
}

resource "aws_subnet" "private" {
  count            = length(var.private_subnet_cidrs)
  vpc_id          = aws_vpc.main.id
  cidr_block      = var.private_subnet_cidrs[count.index]
  availability_zone = var.azs[count.index]
  tags = { Name = "lab4-private-${count.index}" }
}

resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id
  tags = { Name = "lab4-nat" }
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}
```

```

    }
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }
}

resource "aws_route_table_association" "public_assoc" {
  count          = length(aws_subnet.public)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table_association" "private_assoc" {
  count          = length(aws_subnet.private)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private_rt.id
}

```

Step 5 — modules/network/outputs.tf

```

output "vpc_id"          { value = aws_vpc.main.id }
output "public_subnet_ids" { value = aws_subnet.public[*].id }
output "private_subnet_ids" { value = aws_subnet.private[*].id }

```

Step 6 — Root variables.tf

```

variable "db_username" {
  default    = "admin"
  sensitive  = true
}

variable "db_password" {
  description = "Set via TF_VAR_db_password environment variable, do not hardcode"
  sensitive   = true
}

```

Step 7 — Root main.tf — call the module + build the app tier

```

module "network" {
  source = "../modules/network"
}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

resource "aws_security_group" "alb_sg" {

```

```

name    = "lab4-alb-sg"
vpc_id = module.network.vpc_id
ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}
}

resource "aws_security_group" "web_sg" {
    name    = "lab4-web-sg"
    vpc_id = module.network.vpc_id
    ingress {
        from_port      = 80
        to_port        = 80
        protocol        = "tcp"
        security_groups = [aws_security_group.alb_sg.id]
    }
    egress {
        from_port = 0
        to_port   = 0
        protocol  = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

resource "aws_security_group" "db_sg" {
    name    = "lab4-db-sg"
    vpc_id = module.network.vpc_id
    ingress {
        from_port      = 3306
        to_port        = 3306
        protocol        = "tcp"
        security_groups = [aws_security_group.web_sg.id]
    }
    egress {
        from_port = 0
        to_port   = 0
        protocol  = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

resource "aws_lb" "app_alb" {
    name            = "lab4-alb"
    internal        = false
    load_balancer_type = "application"
    security_groups = [aws_security_group.alb_sg.id]
}

```

```

    subnets          = module.network.public_subnet_ids
  }

resource "aws_lb_target_group" "app_tg" {
  name      = "lab4-tg"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = module.network.vpc_id
  health_check {
    path = "/"
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app_alb.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.app_tg.arn
  }
}

resource "aws_launch_template" "web_lt" {
  name_prefix = "lab4-web-"
  image_id    = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"

  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = base64encode(<<-EOF
    #!/bin/bash
    yum update -y
    yum install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo "<h1>Lab 4: Served by $(hostname -f)</h1>" > /var/www/html/index.html
  EOF
)
}

resource "aws_autoscaling_group" "web_asg" {
  desired_capacity = 2
  min_size         = 2
  max_size         = 4
  vpc_zone_identifier = module.network.public_subnet_ids
  target_group_arns   = [aws_lb_target_group.app_tg.arn]

  launch_template {
    id      = aws_launch_template.web_lt.id
    version = "$Latest"
  }

  tag {
    key = "Name"
  }
}

```

```

        value                = "lab4-web-asg"
        propagate_at_launch = true
    }
}

resource "aws_db_subnet_group" "db_subnets" {
    name            = "lab4-db-subnet-group"
    subnet_ids     = module.network.private_subnet_ids
}

resource "aws_db_instance" "app_db" {
    identifier          = "lab4-db"
    engine              = "mysql"
    engine_version     = "8.0"
    instance_class      = "db.t3.micro"
    allocated_storage  = 20
    username            = var.db_username
    password            = var.db_password
    db_subnet_group_name = aws_db_subnet_group.db_subnets.name
    vpc_security_group_ids = [aws_security_group.db_sg.id]
    skip_final_snapshot = true
    publicly_accessible = false
}

```

Step 8 — Root outputs.tf

```

output "alb_dns_name" {
    value = aws_lb.app_alb.dns_name
}

output "db_endpoint" {
    value      = aws_db_instance.app_db.endpoint
    sensitive = true
}

```

Step 9 — Set the DB password securely (never hardcode it)

```
export TF_VAR_db_password="ChangeMe123!"
```

Step 10 — Run the workflow

```

terraform init
terraform fmt -recursive
terraform validate
terraform plan
terraform apply

```

This will take **10–15 minutes**, mostly waiting for the RDS instance to become available.

Step 11 — Validate

```
terraform output alb_dns_name
```

Open <http://<alb-dns-name>> in a browser. Refresh several times — you should see the hostname change as the ALB distributes traffic across the 2 ASG instances (demonstrates load balancing).

2.4.5 Cleanup

```
terraform destroy
```

This takes several minutes (RDS deletion is the slowest step). Confirm in the console that the ALB, ASG, instances, NAT Gateway, EIP, and RDS instance are all gone (NAT Gateways and EIPs incur cost if left behind!).

2.4.6 Challenge / Extension

- Add an `aws_autoscaling_policy` for CPU-based scaling.
- Move the RDS password into AWS Secrets Manager and reference it via a data source instead of a plain variable.
- Split the app tier into its own module, mirroring the network module.

2.4.7 Troubleshooting Notes

- If `apply` fails on the RDS resource with a subnet group AZ error, ensure your two private subnets are in **different** AZs (RDS requires at least 2 AZs in the subnet group).
- If the ALB shows 503 errors, check target group health checks — the ASG instances may still be booting; wait 2–3 minutes.
- Remember to `unset TF_VAR_db_password` after the lab if on a shared machine.

3 Part 2: Azure Labs

3.1 Lab AZ-1: Deploy Your First Linux VM on Azure (Basic)

Difficulty: Basic | **Estimated Time:** 30–40 minutes

3.1.1 Objective

Learn the Terraform + Azure workflow by deploying a resource group, a virtual network, and a Linux VM using the `azurerm` provider.

3.1.2 Prerequisites

- An Azure subscription
- Azure CLI installed (`az --version`)
- Logged in via `az login`

3.1.3 Architecture Overview

- 1 Resource Group
- 1 Virtual Network + 1 Subnet
- 1 Network Interface + 1 Public IP
- 1 Network Security Group allowing SSH (22) and HTTP (80)
- 1 Linux VM (`Standard_B1s`) with a simple web server via `custom_data`

3.1.4 Step-by-Step Instructions

Step 1 — Log in to Azure

```
az login
az account show    # confirm the correct subscription is active
```

Step 2 — Project setup

```
mkdir terraform-azure-lab1 && cd terraform-azure-lab1
```

Step 3 — provider.tf

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.90"
    }
  }
}

provider "azurerm" {
  features {}
}
```

Step 4 — main.tf

```
resource "azurerm_resource_group" "rg" {
  name     = "rg-lab1"
  location = "East US"
}

resource "azurerm_virtual_network" "vnet" {
```

```

    name                = "vnet-lab1"
    address_space        = ["10.0.0.0/16"]
    location              = azurerm_resource_group.rg.location
    resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
    name                = "subnet-lab1"
    resource_group_name = azurerm_resource_group.rg.name
    virtual_network_name = azurerm_virtual_network.vnet.name
    address_prefixes    = ["10.0.1.0/24"]
}

resource "azurerm_public_ip" "pip" {
    name                = "pip-lab1"
    location            = azurerm_resource_group.rg.location
    resource_group_name = azurerm_resource_group.rg.name
    allocation_method   = "Static"
    sku                 = "Standard"
}

resource "azurerm_network_security_group" "nsg" {
    name                = "nsg-lab1"
    location            = azurerm_resource_group.rg.location
    resource_group_name = azurerm_resource_group.rg.name

    security_rule {
        name                = "Allow-SSH"
        priority            = 100
        direction           = "Inbound"
        access              = "Allow"
        protocol            = "Tcp"
        source_port_range   = "*"
        destination_port_range = "22"
        source_address_prefix = "*"
        destination_address_prefix = "*"
    }

    security_rule {
        name                = "Allow-HTTP"
        priority            = 110
        direction           = "Inbound"
        access              = "Allow"
        protocol            = "Tcp"
        source_port_range   = "*"
        destination_port_range = "80"
        source_address_prefix = "*"
        destination_address_prefix = "*"
    }
}

resource "azurerm_network_interface" "nic" {
    name                = "nic-lab1"
    location            = azurerm_resource_group.rg.location
    resource_group_name = azurerm_resource_group.rg.name
}

```

```

ip_configuration {
  name                = "internal"
  subnet_id           = azurerm_subnet.subnet.id
  private_ip_address_allocation = "Dynamic"
  public_ip_address_id = azurerm_public_ip.pip.id
}
}

resource "azurerm_network_interface_security_group_association" "nic_nsg" {
  network_interface_id      = azurerm_network_interface.nic.id
  network_security_group_id = azurerm_network_security_group.nsg.id
}

resource "azurerm_linux_virtual_machine" "vm" {
  name                = "vm-lab1"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  size                = "Standard_B1s"
  admin_username      = "azureuser"
  network_interface_ids = [azurerm_network_interface.nic.id]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("~/ssh/id_rsa.pub")
  }

  os_disk {
    caching          = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  custom_data = base64encode(<<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y apache2
    echo "<h1>Lab 1: Azure VM via Terraform</h1>" > /var/www/html/index.html
    systemctl enable apache2
    systemctl start apache2
    EOF
  )
}

output "public_ip_address" {
  value = azurerm_public_ip.pip.ip_address
}

```

Step 5 — SSH key

If you don't already have one:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa
```

Step 6 — Run the Terraform workflow

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

Type yes.

Step 7 — Validate

```
terraform output public_ip_address
```

Browse to <http://<ip>> — you should see “Lab 1: Azure VM via Terraform”. Optionally SSH in: `ssh azureuser@<ip>`.

3.1.5 Cleanup

```
terraform destroy
```

Confirm in the Azure Portal that `rg-lab1` and all its resources are gone.

3.1.6 Challenge / Extension

- Parametrize `location` and `vm_size` using variables and a `terraform.tfvars` file.
- Add a second NSG rule restricting SSH to your own IP only (`source_address_prefix = "<your-ip>/32"`).

3.1.7 Troubleshooting Notes

- **InvalidParameter: SSH key:** make sure `~/.ssh/id_rsa.pub` exists; generate one if missing.
- **Provider authentication errors:** run `az login` again, or check `az account show` shows the correct subscription/tenant.

3.2 Lab AZ-2: Multi-Tier VNet with NSG Rules and Availability Set (Basic–Moderate)

Difficulty: Basic–Moderate | **Estimated Time:** 45–60 minutes

3.2.1 Objective

Extend the networking model: create a VNet with two subnets (web and data tiers), tier-specific NSGs, and two VMs placed in an Availability Set for basic HA.

3.2.2 Prerequisites

- Completion of Lab AZ-1

3.2.3 Architecture Overview

- 1 Resource Group, 1 VNet (10.1.0.0/16)
- Subnet **web** (10.1.1.0/24) and subnet **data** (10.1.2.0/24)
- NSG on **web** allowing 22/80; NSG on **data** allowing 22 and 1433 (SQL) **only from the web subnet**
- 1 Availability Set with 2 VMs in the web subnet

3.2.4 Step-by-Step Instructions

Step 1 — Project setup

```
mkdir terraform-azure-lab2 && cd terraform-azure-lab2
```

Step 2 — provider.tf (same pattern as Lab AZ-1)

Step 3 — variables.tf

```
variable "location" {  
  default = "East US"  
}
```

```
variable "vm_count" {  
  default = 2  
}
```

Step 4 — network.tf

```
resource "azurerm_resource_group" "rg" {  
  name      = "rg-lab2"  
  location = var.location  
}  
  
resource "azurerm_virtual_network" "vnet" {  
  name            = "vnet-lab2"  
  address_space   = ["10.1.0.0/16"]  
  location        = azurerm_resource_group.rg.location  
  resource_group_name = azurerm_resource_group.rg.name  
}  
  
resource "azurerm_subnet" "web" {  
  name            = "subnet-web"  
  resource_group_name = azurerm_resource_group.rg.name  
  virtual_network_name = azurerm_virtual_network.vnet.name  
  address_prefixes   = ["10.1.1.0/24"]  
}
```

```

resource "azurerm_subnet" "data" {
  name                = "subnet-data"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.1.2.0/24"]
}

resource "azurerm_network_security_group" "web_nsg" {
  name                = "nsg-web-lab2"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  security_rule {
    name                = "Allow-SSH"
    priority            = 100
    direction          = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "22"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
  security_rule {
    name                = "Allow-HTTP"
    priority            = 110
    direction          = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "80"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}

resource "azurerm_network_security_group" "data_nsg" {
  name                = "nsg-data-lab2"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  security_rule {
    name                = "Allow-From-Web-SQL"
    priority            = 100
    direction          = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "1433"
    source_address_prefix = "10.1.1.0/24"
    destination_address_prefix = "*"
  }
}

```

```

resource "azurerm_subnet_network_security_group_association" "web_assoc" {
  subnet_id          = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web_nsg.id
}

resource "azurerm_subnet_network_security_group_association" "data_assoc" {
  subnet_id          = azurerm_subnet.data.id
  network_security_group_id = azurerm_network_security_group.data_nsg.id
}

```

Step 5 — compute.tf

```

resource "azurerm_availability_set" "web_avset" {
  name                  = "avset-web-lab2"
  location              = azurerm_resource_group.rg.location
  resource_group_name   = azurerm_resource_group.rg.name
  platform_fault_domain_count = 2
  platform_update_domain_count = 2
  managed               = true
}

resource "azurerm_public_ip" "web_pip" {
  count                = var.vm_count
  name                = "pip-web-${count.index}"
  location             = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method    = "Static"
  sku                  = "Standard"
}

resource "azurerm_network_interface" "web_nic" {
  count                = var.vm_count
  name                = "nic-web-${count.index}"
  location             = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                  = "internal"
    subnet_id            = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id   = azurerm_public_ip.web_pip[count.index].id
  }
}

resource "azurerm_linux_virtual_machine" "web_vm" {
  count                = var.vm_count
  name                = "vm-web-${count.index}"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  size                = "Standard_B1s"
  admin_username      = "azureuser"
  availability_set_id = azurerm_availability_set.web_avset.id
  network_interface_ids = [azurerm_network_interface.web_nic[count.index].id]

  admin_ssh_key {

```

```

    username    = "azureuser"
    public_key = file("~/ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  custom_data = base64encode(<<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y apache2
    echo "<h1>Lab 2 - Node ${count.index}</h1>" > /var/www/html/index.html
    systemctl enable apache2
    systemctl start apache2
    EOF
  )
}

```

Step 6 — outputs.tf

```

output "web_public_ips" {
  value = azurerm_public_ip.web_pip[*].ip_address
}

```

Step 7 — Run the workflow

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Step 8 — Validate

```

terraform output web_public_ips

```

Browse to each IP and confirm each shows a different node index. In the portal, confirm both VMs sit in different fault domains under the Availability Set.

3.2.5 Cleanup

```

terraform destroy

```

3.2.6 Challenge / Extension

- Add an Azure Load Balancer in front of the two web VMs.
- Add a data-tier VM in the data subnet and prove (via `nc` or `Test-NetConnection`) that port 1433 is reachable only from the web subnet.

3.2.7 Troubleshooting Notes

- **count.index reference errors:** remember array-based resources need `resource[count.index]` syntax when referencing from another `count`-based resource.
- If two VMs land in the same fault domain, that's expected behavior in regions with limited fault domains — check region docs; it does not break the lab.

3.3 Lab AZ-3: Remote State in Azure Storage + App Service with SQL Database (Moderate)

Difficulty: Moderate | **Estimated Time:** 60–75 minutes **Cost note:** Azure SQL Database (Basic tier) and App Service (B1) incur a small hourly charge — destroy promptly.

3.3.1 Objective

Configure an `azurerm` remote backend using Azure Storage, then deploy a PaaS 2-tier application: an Azure App Service (Linux, running a simple app) connected to an Azure SQL Database, with secrets handled via variables (not hardcoded).

3.3.2 Prerequisites

- Completion of Labs AZ-1 and AZ-2

3.3.3 Architecture Overview

- Bootstrap: 1 Resource Group + 1 Storage Account + 1 Blob Container for remote state
- Main stack: 1 App Service Plan (Linux, B1) + 1 App Service (Linux, Node.js) + 1 Azure SQL Server + 1 Azure SQL Database (Basic tier) + firewall rule allowing Azure services

3.3.4 Step-by-Step Instructions

Step 1 — Bootstrap the backend storage

```
mkdir terraform-azure-lab3-bootstrap && cd terraform-azure-lab3-bootstrap
```

main.tf:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.90"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "state_rg" {
  name     = "rg-tfstate-lab3"
  location = "East US"
}

resource "azurerm_storage_account" "state_sa" {
  name                        = "tfstatelab3${random_id.suffix.hex}"
  resource_group_name        = azurerm_resource_group.state_rg.name
  location                   = azurerm_resource_group.state_rg.location
  account_tier               = "Standard"
  account_replication_type   = "LRS"
}

resource "azurerm_storage_container" "state_container" {
  name = "tfstate"
}
```

```

    storage_account_name = azurerm_storage_account.state_sa.name
    container_access_type = "private"
}

resource "random_id" "suffix" {
  byte_length = 4
}

output "storage_account_name" {
  value = azurerm_storage_account.state_sa.name
}

```

```

terraform init
terraform apply

```

Note the `storage_account_name` output.

Step 2 — Main project

```

cd ..
mkdir terraform-azure-lab3 && cd terraform-azure-lab3

```

`backend.tf` (replace storage account name):

```

terraform {
  required_version = ">= 1.6.0"
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.90"
    }
  }

  backend "azurerm" {
    resource_group_name = "rg-tfstate-lab3"
    storage_account_name = "tfstatelab3XXXXXXXXX"
    container_name       = "tfstate"
    key                  = "lab3.terraform.tfstate"
  }
}

provider "azurerm" {
  features {}
}

```

Step 3 — `variables.tf`

```

variable "sql_admin_username" {
  default = "sqladmin"
}

variable "sql_admin_password" {
  description = "Set via TF_VAR_sql_admin_password"
  sensitive   = true
}

```

Step 4 — `main.tf`

```

resource "azurerm_resource_group" "rg" {
  name      = "rg-lab3"
  location  = "East US"
}

resource "azurerm_service_plan" "app_plan" {
  name                = "asp-lab3"
  resource_group_name = azurerm_resource_group.rg.name
  location             = azurerm_resource_group.rg.location
  os_type              = "Linux"
  sku_name             = "B1"
}

resource "azurerm_linux_web_app" "app" {
  name                = "lab3-webapp-${random_id.app_suffix.hex}"
  resource_group_name = azurerm_resource_group.rg.name
  location             = azurerm_resource_group.rg.location
  service_plan_id     = azurerm_service_plan.app_plan.id

  site_config {
    application_stack {
      node_version = "18-lts"
    }
  }

  app_settings = {
    "SQL_CONNECTION_STRING" = "Server=tcp:${azurerm_mssql_server.sql.fully_qualified_domain_name},1433;Data Source=lab3db;Initial Catalog=lab3db;Integrated Security=true;"
  }
}

resource "random_id" "app_suffix" {
  byte_length = 4
}

resource "azurerm_mssql_server" "sql" {
  name                = "lab3-sqlserver-${random_id.app_suffix.hex}"
  resource_group_name = azurerm_resource_group.rg.name
  location             = azurerm_resource_group.rg.location
  version              = "12.0"
  administrator_login  = var.sql_admin_username
  administrator_login_password = var.sql_admin_password
}

resource "azurerm_mssql_database" "db" {
  name      = "lab3db"
  server_id = azurerm_mssql_server.sql.id
  sku_name  = "Basic"
  max_size_gb = 2
}

resource "azurerm_mssql_firewall_rule" "allow_azure" {
  name      = "AllowAzureServices"
  server_id = azurerm_mssql_server.sql.id
  start_ip_address = "0.0.0.0"
}

```

```
end_ip_address = "0.0.0.0"
}
```

Step 5 — outputs.tf

```
output "web_app_url" {
  value = "https://${azurermlinux_web_app.app.default_hostname}"
}

output "sql_server_fqdn" {
  value = azurermmssql_server.sql.fully_qualified_domain_name
}
```

Step 6 — Set the SQL password securely

```
export TF_VAR_sql_admin_password="ChangeMe123!Strong"
```

Step 7 — Run the workflow

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

Step 8 — Validate

```
terraform output web_app_url
```

Browse to the URL — the default App Service placeholder page should load (a full app deployment is outside this lab's scope, but the plumbing is proven). Confirm in the portal that the SQL Database and App Service both exist and the firewall rule is present.

3.3.5 Cleanup

```
terraform destroy
cd ../terraform-azure-lab3-bootstrap
terraform destroy
```

3.3.6 Challenge / Extension

- Deploy actual application code to the App Service using `az webapp deploy` or a CI/CD pipeline.
- Replace the plaintext password variable with a reference to Azure Key Vault using the `azurermm_key_vault_secret` data source.

3.3.7 Troubleshooting Notes

- **SQL password complexity errors:** Azure SQL requires at least 8 characters with 3 of 4 character classes (upper, lower, digit, symbol).
- **azurermm backend auth errors:** the backend uses your `az login` session by default; if it fails, add `use_azuread_auth = true` or pass a storage access key explicitly.

3.4 Lab AZ-4: AKS (Azure Kubernetes Service) Cluster with Terraform (Advanced)

Difficulty: Advanced | **Estimated Time:** 75–100 minutes **Cost note:** AKS control plane is free, but worker nodes (VMs) incur cost. Use 2 x Standard_B2s nodes and destroy promptly.

3.4.1 Objective

Provision a managed Kubernetes cluster (AKS) using Terraform, configure `kubectl` access via Terraform outputs, and deploy a sample workload to prove the cluster works — the standard entry point into container orchestration with IaC.

3.4.2 Prerequisites

- Completion of Labs AZ-1 through AZ-3
- `kubectl` installed locally
- Basic familiarity with Kubernetes concepts (pods, deployments, services)

3.4.3 Architecture Overview

- 1 Resource Group
- 1 AKS Cluster with a default node pool (2 nodes, Standard_B2s), using the Azure CNI or Kubenet network plugin (Kubenet used here for simplicity)
- Terraform outputs the `kube_config` used to populate `~/.kube/config`

3.4.4 Step-by-Step Instructions

Step 1 — Project setup

```
mkdir terraform-azure-lab4 && cd terraform-azure-lab4
```

Step 2 — provider.tf

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.90"
    }
  }
}

provider "azurerm" {
  features {}
}
```

Step 3 — main.tf

```
resource "azurerm_resource_group" "rg" {
  name     = "rg-lab4-aks"
  location = "East US"
}

resource "azurerm_kubernetes_cluster" "aks" {
  name                = "aks-lab4"
  location             = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  dns_prefix          = "lab4aks"
}
```

```

default_node_pool {
  name          = "default"
  node_count    = 2
  vm_size       = "Standard_B2s"
}

identity {
  type = "SystemAssigned"
}

network_profile {
  network_plugin = "kubenet"
}

tags = {
  Environment = "lab4"
}
}

```

Step 4 — outputs.tf

```

output "kube_config" {
  value      = azurerm_kubernetes_cluster.aks.kube_config_raw
  sensitive = true
}

output "cluster_name" {
  value = azurerm_kubernetes_cluster.aks.name
}

```

Step 5 — Run the workflow

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

This takes **8–12 minutes** — AKS provisioning is slow.

Step 6 — Configure kubectl

```

terraform output -raw kube_config > ./kubeconfig_lab4
export KUBECONFIG=./kubeconfig_lab4
kubectl get nodes

```

You should see 2 nodes in Ready status.

Step 7 — Deploy a sample workload

Create nginx-deploy.yaml:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-lab4
spec:
  replicas: 2
  selector:

```

```

    matchLabels:
      app: nginx-lab4
  template:
    metadata:
      labels:
        app: nginx-lab4
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-lab4-svc
spec:
  type: LoadBalancer
  selector:
    app: nginx-lab4
  ports:
  - port: 80
    targetPort: 80

```

```

kubectl apply -f nginx-deploy.yaml
kubectl get svc nginx-lab4-svc --watch

```

Wait for EXTERNAL-IP to populate (a couple of minutes — Azure provisions a Load Balancer). Browse to `http://<external-ip>` and confirm the NGINX welcome page loads.

3.4.5 Cleanup

```

kubectl delete -f nginx-deploy.yaml    # deletes the LoadBalancer Service FIRST, releasing the Azure LB
terraform destroy

```

Important: always delete Kubernetes LoadBalancer-type Services before running `terraform destroy` — otherwise Terraform may not clean up the underlying Azure Load Balancer resource it doesn’t manage directly, leaving orphaned billing resources. Confirm no leftover Load Balancer or Public IP remains in the resource group afterward.

3.4.6 Challenge / Extension

- Enable the Kubernetes Cluster Autoscaler (`auto_scaling_enabled = true` with `min_count/max_count` on the node pool).
- Add a second node pool for a different workload class (e.g., spot instances for batch jobs).
- Integrate `azurerm_role_assignment` to grant the AKS managed identity pull access to an Azure Container Registry.

3.4.7 Troubleshooting Notes

- **kubectl “Unable to connect to the server”:** confirm KUBECONFIG env var points to the right file, and that the file wasn’t truncated (`-raw` flag is required on `terraform output`).
- If node provisioning fails with quota errors, request a quota increase for **Standard_B2s** in your subscription/region, or switch to a smaller size supported in your region.
- AKS deletion can take 5–10 minutes; this is normal.

4 Part 3: GCP Labs

4.1 Lab GCP-1: Deploy Your First Compute Engine VM (Basic)

Difficulty: Basic | **Estimated Time:** 30–40 minutes

4.1.1 Objective

Learn the Terraform + GCP workflow by deploying a single Compute Engine VM with a firewall rule allowing HTTP/SSH.

4.1.2 Prerequisites

- A GCP project with billing enabled
- gcloud CLI installed and authenticated
- The Compute Engine API enabled on the project

4.1.3 Architecture Overview

- 1 firewall rule allowing inbound TCP 22 and 80
- 1 e2-micro Compute Engine instance running Debian, with a startup script installing Apache

4.1.4 Step-by-Step Instructions

Step 1 — Authenticate and set your project

```
gcloud auth login
gcloud auth application-default login
gcloud config set project YOUR_PROJECT_ID
gcloud services enable compute.googleapis.com
```

Step 2 — Project setup

```
mkdir terraform-gcp-lab1 && cd terraform-gcp-lab1
```

Step 3 — provider.tf

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    google = {
      source = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "google" {
  project = "YOUR_PROJECT_ID"
  region  = "us-central1"
  zone    = "us-central1-a"
}
```

Step 4 — main.tf

```
resource "google_compute_firewall" "allow_web_ssh" {
  name     = "lab1-allow-web-ssh"
  network  = "default"

  allow {
```

```

    protocol = "tcp"
    ports    = ["22", "80"]
  }

  source_ranges = ["0.0.0.0/0"]
}

resource "google_compute_instance" "web" {
  name          = "lab1-web-vm"
  machine_type  = "e2-micro"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
    }
  }

  network_interface {
    network = "default"
    access_config {} # ephemeral public IP
  }

  metadata_startup_script = <<-EOF
    #!/bin/bash
    apt-get update
    apt-get install -y apache2
    echo "<h1>Lab 1: GCP Compute Engine via Terraform</h1>" > /var/www/html/index.html
    systemctl enable apache2
    systemctl restart apache2
  EOF

  tags = ["lab1-web"]
}

output "instance_public_ip" {
  value = google_compute_instance.web.network_interface[0].access_config[0].nat_ip
}

```

Step 5 — Run the workflow

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Step 6 — Validate

```
terraform output instance_public_ip
```

Browse to <http://<ip>> — you should see the Lab 1 page (allow ~30–60 seconds after apply for the startup script to finish).

4.1.5 Cleanup

```
terraform destroy
```

Confirm in the GCP Console (Compute Engine → VM instances) that the instance is gone.

4.1.6 Challenge / Extension

- Parametrize `machine_type` and `image` as variables.
- Restrict the firewall's `source_ranges` to your own IP for SSH.

4.1.7 Troubleshooting Notes

- **Error 403: Compute Engine API has not been used:** `runcloud services enable compute.googleapis.com` and wait a minute for propagation.
- **Quota errors on e2-micro:** check the free-tier region restrictions — `e2-micro` free tier is limited to specific US regions.

4.2 Lab GCP-2: Custom VPC with Subnet, Firewall Rules, and Multiple Instances (Basic–Moderate)

Difficulty: Basic–Moderate | **Estimated Time:** 45–60 minutes

4.2.1 Objective

Move off the `default` network and build a custom VPC with a purpose-built subnet, tag-based firewall rules, and two instances behind it, using `count` for repeatable infrastructure.

4.2.2 Prerequisites

- Completion of Lab GCP-1

4.2.3 Architecture Overview

- 1 custom VPC (auto-subnet mode disabled) + 1 custom subnet (10.10.0.0/24) in `us-central1`
- Firewall rules: allow internal traffic within the subnet; allow external SSH (22) and HTTP (80) only on instances tagged `web`
- 2 Compute Engine instances (`e2-micro`) in the subnet

4.2.4 Step-by-Step Instructions

Step 1 — Project setup

```
mkdir terraform-gcp-lab2 && cd terraform-gcp-lab2
```

Step 2 — `provider.tf` (same pattern as Lab GCP-1, update project ID)

Step 3 — `variables.tf`

```
variable "project_id" {
  description = "Your GCP project ID"
}

variable "region" {
  default = "us-central1"
}

variable "zone" {
  default = "us-central1-a"
}

variable "instance_count" {
  default = 2
}
```

Step 4 — `network.tf`

```
resource "google_compute_network" "vpc" {
  name                = "lab2-vpc"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "subnet" {
  name                = "lab2-subnet"
  ip_cidr_range       = "10.10.0.0/24"
  region              = var.region
  network              = google_compute_network.vpc.id
}
```

```

}

resource "google_compute_firewall" "allow_internal" {
  name      = "lab2-allow-internal"
  network   = google_compute_network.vpc.id

  allow {
    protocol = "tcp"
    ports    = ["0-65535"]
  }
  allow {
    protocol = "udp"
    ports    = ["0-65535"]
  }
  allow {
    protocol = "icmp"
  }

  source_ranges = ["10.10.0.0/24"]
}

resource "google_compute_firewall" "allow_web_ssh" {
  name      = "lab2-allow-web-ssh"
  network   = google_compute_network.vpc.id

  allow {
    protocol = "tcp"
    ports    = ["22", "80"]
  }

  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["web"]
}

```

Step 5 — compute.tf

```

resource "google_compute_instance" "web" {
  count          = var.instance_count
  name           = "lab2-web-vm-${count.index}"
  machine_type   = "e2-micro"
  zone           = var.zone
  tags           = ["web"]

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
    }
  }

  network_interface {
    network      = google_compute_network.vpc.id
    subnetwork   = google_compute_subnetwork.subnet.id
    access_config {}
  }
}

```

```

metadata_startup_script = <<-EOF
#!/bin/bash
apt-get update
apt-get install -y apache2
echo "<h1>Lab 2 - Node ${count.index}</h1>" > /var/www/html/index.html
systemctl enable apache2
systemctl restart apache2
EOF
}

```

Step 6 — outputs.tf

```

output "instance_ips" {
  value = [for inst in google_compute_instance.web : inst.network_interface[0].access_config[0].nat_ip]
}

```

Step 7 — terraform.tfvars

```

project_id = "YOUR_PROJECT_ID"

```

Step 8 — Update provider.tf to use the variable

```

provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}

```

Step 9 — Run the workflow

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Step 10 — Validate

```

terraform output instance_ips

```

Browse to each IP and confirm each shows its own node index. In the console, confirm both instances live in the custom subnet, not **default**.

4.2.5 Cleanup

```

terraform destroy

```

4.2.6 Challenge / Extension

- Add a Cloud NAT + Cloud Router so a third instance can be created with **no** external IP but still reach the internet outbound.
- Convert the firewall rules to use service accounts instead of tags for identity-based rules.

4.2.7 Troubleshooting Notes

- **for expression errors in outputs:** GCP's `access_config` is a list, so always index `[0]` even when only one config block exists.
- If instances can't ping each other, verify the `allow_internal` rule's `source_ranges` matches your subnet CIDR exactly.

4.3 Lab GCP-3: Remote State in GCS + Cloud SQL Backend (Moderate)

Difficulty: Moderate | **Estimated Time:** 60–75 minutes **Cost note:** Cloud SQL (db-f1-micro) incurs a small hourly charge — destroy promptly.

4.3.1 Objective

Configure a GCS remote backend for Terraform state, then provision a Cloud SQL (PostgreSQL) instance with private/authorized-network access, plus a Compute Engine instance configured to connect to it — the classic GCP 2-tier pattern.

4.3.2 Prerequisites

- Completion of Labs GCP-1 and GCP-2

4.3.3 Architecture Overview

- Bootstrap: 1 GCS bucket (versioned) to hold Terraform state
- Main stack: 1 Cloud SQL PostgreSQL instance (db-f1-micro) with an authorized network, 1 database, 1 user, and 1 Compute Engine instance with the Cloud SQL connection details injected via startup script metadata

4.3.4 Step-by-Step Instructions

Step 1 — Bootstrap the GCS backend bucket

```
mkdir terraform-gcp-lab3-bootstrap && cd terraform-gcp-lab3-bootstrap
```

main.tf:

```
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "google" {
  project = "YOUR_PROJECT_ID"
  region  = "us-central1"
}

resource "random_id" "suffix" {
  byte_length = 4
}

resource "google_storage_bucket" "tf_state" {
  name                  = "tf-state-lab3-${random_id.suffix.hex}"
  location              = "US"
  uniform_bucket_level_access = true

  versioning {
    enabled = true
  }
}
```

```
output "bucket_name" {
  value = google_storage_bucket.tf_state.name
}
```

```
terraform init
terraform apply
```

Step 2 — Main project

```
cd ..
mkdir terraform-gcp-lab3 && cd terraform-gcp-lab3
```

backend.tf (replace bucket name):

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }

  backend "gcs" {
    bucket = "tf-state-lab3-XXXXXXX"
    prefix = "lab3/state"
  }
}

provider "google" {
  project = "YOUR_PROJECT_ID"
  region  = "us-central1"
  zone    = "us-central1-a"
}
```

Step 3 — variables.tf

```
variable "db_password" {
  description = "Set via TF_VAR_db_password"
  sensitive   = true
}
```

Step 4 — main.tf

```
resource "google_sql_database_instance" "pg" {
  name             = "lab3-pg-instance"
  database_version = "POSTGRES_15"
  region           = "us-central1"

  settings {
    tier = "db-f1-micro"

    ip_configuration {
      ipv4_enabled = true
      authorized_networks {
        name = "allow-all-lab-demo"
        value = "0.0.0.0/0"
      }
    }
  }
}
```

```

    }
  }

  deletion_protection = false
}

resource "google_sql_database" "app_db" {
  name      = "lab3appdb"
  instance = google_sql_database_instance.pg.name
}

resource "google_sql_user" "app_user" {
  name      = "labuser"
  instance = google_sql_database_instance.pg.name
  password = var.db_password
}

resource "google_compute_firewall" "allow_ssh_http" {
  name      = "lab3-allow-ssh-http"
  network = "default"

  allow {
    protocol = "tcp"
    ports    = ["22", "80"]
  }

  source_ranges = ["0.0.0.0/0"]
}

resource "google_compute_instance" "app" {
  name          = "lab3-app-vm"
  machine_type = "e2-micro"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
    }
  }

  network_interface {
    network = "default"
    access_config {}
  }

  metadata_startup_script = <<-EOF
    #!/bin/bash
    apt-get update
    apt-get install -y postgresql-client apache2
    echo "<h1>Lab 3: App VM connected to Cloud SQL ${google_sql_database_instance.pg.connection_name}</h1>"
    systemctl enable apache2
    systemctl restart apache2
  EOF
}

```

Step 5 — outputs.tf

```
output "db_public_ip" {
  value = google_sql_database_instance.pg.public_ip_address
}

output "app_vm_ip" {
  value = google_compute_instance.app.network_interface[0].access_config[0].nat_ip
}
```

Step 6 — Set the DB password securely

```
export TF_VAR_db_password="ChangeMe123!"
```

Step 7 — Run the workflow

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

Cloud SQL provisioning takes **5–10 minutes**.

Step 8 — Validate

```
terraform output app_vm_ip
terraform output db_public_ip
```

Browse http://<app_vm_ip> and confirm the page displays the Cloud SQL connection name. Optionally, from the VM (via SSH), test:

```
psql "host=<db_public_ip> port=5432 sslmode=disable dbname=lab3appdb user=labuser"
```

4.3.5 Cleanup

```
terraform destroy
cd ../terraform-gcp-lab3-bootstrap
terraform destroy
```

4.3.6 Challenge / Extension

- Replace the 0.0.0.0/0 authorized network with the Cloud SQL Auth Proxy for secure, IP-less connections.
- Move db_password into Google Secret Manager and reference it via a data source.

4.3.7 Troubleshooting Notes

- **Error 409: The Cloud SQL instance already exists:** Cloud SQL instance names are globally unique per project *and reserved for a week after deletion* — always use a unique suffix if re-running the lab.
- If psql connection times out, confirm the authorized network includes your test source IP, and that `ipv4_enabled = true`.

4.4 Lab GCP-4: GKE (Google Kubernetes Engine) Cluster with Modules (Advanced)

Difficulty: Advanced | **Estimated Time:** 75–100 minutes **Cost note:** GKE Autopilot/Standard node VMs incur cost — use 2 x e2-medium nodes and destroy promptly. (GKE cluster management fee may apply outside the free tier for additional zonal clusters.)

4.4.1 Objective

Provision a GKE Standard cluster using a reusable local module, configure `kubectl` access, and deploy a sample workload — mirroring the AKS lab so students can directly compare the two clouds' patterns.

4.4.2 Prerequisites

- Completion of Labs GCP-1 through GCP-3
- `kubectl` and `gke-gcloud-auth-plugin` installed (`gcloud components install gke-gcloud-auth-plugin`)

4.4.3 Architecture Overview

- A local module `modules/gke_cluster` encapsulating the cluster + node pool
- Root module calling it with lab-specific variables
- 1 GKE cluster (regional not required — zonal for cost savings) with 1 node pool of 2 e2-medium nodes

4.4.4 Step-by-Step Instructions

Step 1 — Project structure

```
mkdir -p terraform-gcp-lab4/modules/gke_cluster
cd terraform-gcp-lab4
```

Step 2 — Module — `modules/gke_cluster/variables.tf`

```
variable "cluster_name" {}
variable "zone"          {}
variable "node_count"    { default = 2 }
variable "machine_type"  { default = "e2-medium" }
```

Step 3 — `modules/gke_cluster/main.tf`

```
resource "google_container_cluster" "primary" {
  name     = var.cluster_name
  location = var.zone

  remove_default_node_pool = true
  initial_node_count       = 1
}

resource "google_container_node_pool" "primary_nodes" {
  name        = "${var.cluster_name}-node-pool"
  location    = var.zone
  cluster     = google_container_cluster.primary.name
  node_count  = var.node_count

  node_config {
    machine_type = var.machine_type
    oauth_scopes = [
      "https://www.googleapis.com/auth/cloud-platform"
    ]
  }
}
```

```
}  
}
```

Step 4 — modules/gke_cluster/outputs.tf

```
output "cluster_name"      { value = google_container_cluster.primary.name }  
output "cluster_endpoint" { value = google_container_cluster.primary.endpoint }  
output "cluster_ca_cert"  { value = google_container_cluster.primary.master_auth[0].cluster_ca_certificate }
```

Step 5 — Root provider.tf

```
terraform {  
  required_version = ">= 1.6.0"  
  required_providers {  
    google = {  
      source  = "hashicorp/google"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "google" {  
  project = "YOUR_PROJECT_ID"  
  region  = "us-central1"  
  zone    = "us-central1-a"  
}
```

Step 6 — Root main.tf

```
module "gke" {  
  source      = "./modules/gke_cluster"  
  cluster_name = "lab4-gke"  
  zone        = "us-central1-a"  
  node_count  = 2  
  machine_type = "e2-medium"  
}
```

Step 7 — Root outputs.tf

```
output "cluster_name" {  
  value = module.gke.cluster_name  
}  
  
output "cluster_endpoint" {  
  value = module.gke.cluster_endpoint  
}
```

Step 8 — Run the workflow

```
terraform init  
terraform fmt -recursive  
terraform validate  
terraform plan  
terraform apply
```

This takes **6–10 minutes**.

Step 9 — Configure kubectl

```
gcloud container clusters get-credentials lab4-gke --zone us-central1-a --project YOUR_PROJECT_ID
kubectl get nodes
```

You should see 2 nodes Ready.

Step 10 — Deploy a sample workload

Create `nginx-deploy.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-lab4
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-lab4
  template:
    metadata:
      labels:
        app: nginx-lab4
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-lab4-svc
spec:
  type: LoadBalancer
  selector:
    app: nginx-lab4
  ports:
  - port: 80
    targetPort: 80
```

```
kubectl apply -f nginx-deploy.yaml
kubectl get svc nginx-lab4-svc --watch
```

Wait for EXTERNAL-IP, then browse to it and confirm NGINX loads.

4.4.5 Cleanup

```
kubectl delete -f nginx-deploy.yaml    # release the GCP Load Balancer first
terraform destroy
```

As with AKS, always delete `LoadBalancer` Services before `terraform destroy` to avoid orphaned GCP forwarding rules and external IPs.

4.4.6 Challenge / Extension

- Convert to GKE Autopilot mode and compare the Terraform resource differences.

- Add Workload Identity so pods can securely call other GCP services without service account keys.
- Parametrize the module further to spin up a second, smaller cluster in another region for a DR comparison.

4.4.7 Troubleshooting Notes

- **kubect1 auth plugin errors:** install `gke-gcloud-auth-plugin` and set `export USE_GKE_GCLOUD_AUTH_PLUGIN=True`.
- If `terraform destroy` fails because the node pool is “in use,” delete Kubernetes-created load balancer resources first (Step “Cleanup” above), then retry.
- GKE cluster deletion can take several minutes; this is expected.

5 Part 4: Multi-Cloud Labs

These labs combine two or more providers in a **single Terraform configuration or coordinated set of configurations**. They assume completion of at least the Basic and Moderate labs in each relevant cloud track.

5.1 Lab MC-1: Provision Storage Simultaneously on AWS, Azure, and GCP (Basic)

Difficulty: Basic | **Estimated Time:** 45–60 minutes

5.1.1 Objective

Learn the fundamental multi-cloud pattern: **declaring multiple required_providers in one configuration** and creating equivalent storage resources (an S3 bucket, an Azure Storage Account + Container, and a GCS bucket) in a single `terraform apply`.

5.1.2 Prerequisites

- Valid credentials configured for AWS (`aws configure`), Azure (`az login`), and GCP (`gcloud auth application-default login`)
- Completion of at least the Basic lab in each individual cloud

5.1.3 Architecture Overview

- 1 S3 bucket (AWS) with versioning enabled
- 1 Resource Group + Storage Account + Blob Container (Azure)
- 1 GCS bucket (GCP) with versioning enabled
- All three declared and applied from one root module

5.1.4 Step-by-Step Instructions

Step 1 — Project setup

```
mkdir terraform-multicloud-lab1 && cd terraform-multicloud-lab1
```

Step 2 — provider.tf — declare all three providers

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.90"
    }
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
```

```

provider "aws" {
  region = "us-east-1"
}

provider "azurerm" {
  features {}
}

provider "google" {
  project = "YOUR_GCP_PROJECT_ID"
  region  = "us-central1"
}

```

Step 3 — main.tf

```

resource "random_id" "suffix" {
  byte_length = 4
}

# ----- AWS -----
resource "aws_s3_bucket" "aws_bucket" {
  bucket = "mc-lab1-aws-${random_id.suffix.hex}"
}

resource "aws_s3_bucket_versioning" "aws_bucket_versioning" {
  bucket = aws_s3_bucket.aws_bucket.id
  versioning_configuration {
    status = "Enabled"
  }
}

# ----- Azure -----
resource "azurerm_resource_group" "azure_rg" {
  name      = "rg-mc-lab1"
  location  = "East US"
}

resource "azurerm_storage_account" "azure_sa" {
  name                        = "mclab1sa${random_id.suffix.hex}"
  resource_group_name        = azurerm_resource_group.azure_rg.name
  location                   = azurerm_resource_group.azure_rg.location
  account_tier               = "Standard"
  account_replication_type   = "LRS"
}

resource "azurerm_storage_container" "azure_container" {
  name                = "mc-lab1-container"
  storage_account_name = azurerm_storage_account.azure_sa.name
  container_access_type = "private"
}

# ----- GCP -----
resource "google_storage_bucket" "gcp_bucket" {
  name        = "mc-lab1-gcp-${random_id.suffix.hex}"
  location    = "US"
}

```

```

    uniform_bucket_level_access = true
    versioning {
      enabled = true
    }
  }
}

```

Step 4 — outputs.tf

```

output "aws_bucket_name" {
  value = aws_s3_bucket.aws_bucket.bucket
}

output "azure_storage_account_name" {
  value = azurerm_storage_account.azure_sa.name
}

output "gcp_bucket_name" {
  value = google_storage_bucket.gcp_bucket.name
}

```

Step 5 — Run the workflow

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Notice the plan shows resources tagged `aws_*`, `azurerm_*`, and `google_*` all in one plan — Terraform is provider-agnostic at its core; each provider is just a plugin.

Step 6 — Validate

```
terraform output
```

Check each cloud console to confirm the bucket/storage account was created as expected.

5.1.5 Cleanup

```
terraform destroy
```

This tears down all three clouds’ resources in one command — the core value proposition of a unified IaC tool.

5.1.6 Challenge / Extension

- Upload the same file (e.g., `index.html`) to all three buckets using `aws_s3_object`, `azurerm_storage_blob`, and `google_storage_bucket_object` resources.
- Add tags/labels consistently across all three (note: AWS/GCP call them “tags”/“labels”, Azure calls them “tags” too, but the key-value schemas differ slightly).

5.1.7 Troubleshooting Notes

- **Provider block ordering doesn’t matter** — Terraform resolves the dependency graph regardless of the order providers/resources appear in the file.
- If one cloud’s `apply` fails, the others may still succeed — Terraform reports partial failures resource-by-resource, not all-or-nothing across providers.

5.2 Lab MC-2: Consistent VM Deployment Across AWS, Azure, and GCP Using a Shared Variable Pattern (Moderate)

Difficulty: Moderate | **Estimated Time:** 60–90 minutes

5.2.1 Objective

Deploy an equivalent small web server VM on all three clouds from a single configuration, using a shared `locals` block to keep naming, tags, and user-data logic consistent — demonstrating how teams standardize multi-cloud deployments without three separate codebases.

5.2.2 Prerequisites

- Completion of Lab MC-1
- Completion of the Basic/Moderate labs in each individual cloud track

5.2.3 Architecture Overview

- AWS: 1 EC2 instance + security group (public subnet of default VPC)
- Azure: 1 resource group + VNet/subnet + NSG + Linux VM
- GCP: 1 firewall rule + Compute Engine instance
- A shared `locals` block drives the project name, environment tag, and the HTML content each server displays

5.2.4 Step-by-Step Instructions

Step 1 — Project setup

```
mkdir terraform-multicloud-lab2 && cd terraform-multicloud-lab2
```

Step 2 — `provider.tf` (same three-provider pattern as Lab MC-1, plus GCP zone)

Step 3 — `locals.tf` — the shared configuration

```
locals {
  project_name = "mc-lab2"
  environment  = "training"

  common_tags = {
    Project      = local.project_name
    Environment  = local.environment
    ManagedBy    = "Terraform"
  }

  web_page_html = "<h1>Multi-Cloud Lab 2 - ${local.project_name}</h1>"
}
```

Step 4 — `aws.tf`

```
data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name     = "vpc-id"
    values  = [data.aws_vpc.default.id]
  }
}
```

```

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

resource "aws_security_group" "web_sg" {
  name     = "${local.project_name}-aws-sg"
  vpc_id   = data.aws_vpc.default.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = local.common_tags
}

resource "aws_instance" "web" {
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = "t2.micro"
  subnet_id          = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    yum install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo '${local.web_page_html} (AWS)' > /var/www/html/index.html
  EOF

  tags = merge(local.common_tags, { Name = "${local.project_name}-aws-vm" })
}

```

Step 5 — azure.tf

```

resource "azurerm_resource_group" "rg" {
  name      = "rg-${local.project_name}"
  location  = "East US"
  tags      = local.common_tags
}

resource "azurerm_virtual_network" "vnet" {
  name            = "${local.project_name}-vnet"
  address_space   = ["10.20.0.0/16"]
  location        = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
  name                = "${local.project_name}-subnet"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.20.1.0/24"]
}

resource "azurerm_public_ip" "pip" {
  name            = "${local.project_name}-pip"
  location        = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method = "Static"
  sku             = "Standard"
}

resource "azurerm_network_security_group" "nsg" {
  name            = "${local.project_name}-nsg"
  location        = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  security_rule {
    name                = "Allow-HTTP-SSH"
    priority             = 100
    direction           = "Inbound"
    access               = "Allow"
    protocol             = "Tcp"
    source_port_range    = "*"
    destination_port_ranges = ["22", "80"]
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}

resource "azurerm_network_interface" "nic" {
  name            = "${local.project_name}-nic"
  location        = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                = "internal"
    subnet_id           = azurerm_subnet.subnet.id
  }
}

```

```

        private_ip_address_allocation = "Dynamic"
        public_ip_address_id          = azurerm_public_ip.pip.id
    }
}

resource "azurerm_network_interface_security_group_association" "nic_nsg" {
    network_interface_id      = azurerm_network_interface.nic.id
    network_security_group_id = azurerm_network_security_group.nsg.id
}

resource "azurerm_linux_virtual_machine" "vm" {
    name                = "${local.project_name}-vm"
    resource_group_name = azurerm_resource_group.rg.name
    location            = azurerm_resource_group.rg.location
    size               = "Standard_B1s"
    admin_username     = "azureuser"
    network_interface_ids = [azurerm_network_interface.nic.id]
    tags               = local.common_tags

    admin_ssh_key {
        username   = "azureuser"
        public_key = file("~/ssh/id_rsa.pub")
    }

    os_disk {
        caching              = "ReadWrite"
        storage_account_type = "Standard_LRS"
    }

    source_image_reference {
        publisher = "Canonical"
        offer     = "0001-com-ubuntu-server-jammy"
        sku       = "22_04-lts"
        version   = "latest"
    }

    custom_data = base64encode(<<-EOF
        #!/bin/bash
        apt-get update -y
        apt-get install -y apache2
        echo '${local.web_page_html} (Azure)' > /var/www/html/index.html
        systemctl enable apache2
        systemctl start apache2
    EOF
    )
}

```

Step 6 — gcp.tf

```

resource "google_compute_firewall" "allow_web_ssh" {
    name     = "${local.project_name}-gcp-fw"
    network = "default"

    allow {
        protocol = "tcp"
    }
}

```

```

    ports      = ["22", "80"]
  }

  source_ranges = ["0.0.0.0/0"]
}

resource "google_compute_instance" "web" {
  name          = "${local.project_name}-gcp-vm"
  machine_type  = "e2-micro"
  zone          = "us-central1-a"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
    }
  }

  network_interface {
    network = "default"
    access_config {}
  }

  metadata_startup_script = <<-EOF
    #!/bin/bash
    apt-get update
    apt-get install -y apache2
    echo '${local.web_page_html} (GCP)' > /var/www/html/index.html
    systemctl enable apache2
    systemctl restart apache2
  EOF

  labels = {
    project      = local.project_name
    environment = local.environment
  }
}

```

Step 7 — outputs.tf

```

output "aws_vm_ip"    { value = aws_instance.web.public_ip }
output "azure_vm_ip"  { value = azurerm_public_ip.pip.ip_address }
output "gcp_vm_ip"    { value = google_compute_instance.web.network_interface[0].access_config[0].nat_ip }

```

Step 8 — Run the workflow

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Step 9 — Validate

```

terraform output

```

Browse to all three IPs. Each should show the same base message with a different cloud name appended — proving a single Terraform run produced consistent, comparable infrastructure across three providers.

5.2.5 Cleanup

```
terraform destroy
```

5.2.6 Challenge / Extension

- Replace the three `.tf` files with three **modules** (`modules/aws_vm`, `modules/azure_vm`, `modules/gcp_vm`) sharing a common `variables.tf` interface (e.g., `vm_name`, `tags`).
- Add a `for_each` over a map of “environments” (`dev/test`) to deploy the same trio twice with different names.

5.2.7 Troubleshooting Notes

- Keep in mind each cloud has **different tagging rules** (e.g., GCP labels must be lowercase, no spaces) — the `merge()/labels` blocks above already account for this, but watch for it when extending.
- If one provider’s `apply` fails partway through, re-running `terraform apply` will safely continue only the unfinished resources (Terraform’s state-driven design).

5.3 Lab MC-3: Multi-Cloud Networking with Planned CIDR Ranges and Cross-Cloud Outputs (Moderate–Advanced)

Difficulty: Moderate–Advanced | **Estimated Time:** 75–90 minutes

5.3.1 Objective

Design non-overlapping CIDR ranges across AWS, Azure, and GCP networks (a real prerequisite for any future cross-cloud VPN/peering), build the three networks in one configuration, and produce a single consolidated output showing the full multi-cloud network map — a common architecture review deliverable.

5.3.2 Prerequisites

- Completion of Lab MC-2
- Completion of the Moderate networking labs (AWS-2, AZ-2, GCP-2)

5.3.3 Architecture Overview

- AWS VPC: 10.100.0.0/16 with 1 public subnet 10.100.1.0/24
- Azure VNet: 10.200.0.0/16 with 1 subnet 10.200.1.0/24
- GCP VPC: custom-mode with 1 subnet 10.210.1.0/24 in `us-central1`
- No instances in this lab — pure networking, to keep the focus and cost minimal
- A single `outputs.tf` renders a consolidated CIDR map for documentation purposes

5.3.4 Step-by-Step Instructions

Step 1 — Project setup

```
mkdir terraform-multicloud-lab3 && cd terraform-multicloud-lab3
```

Step 2 — `provider.tf` (three-provider pattern, as in Lab MC-1)

Step 3 — `variables.tf` — the CIDR plan

```
variable "aws_vpc_cidr" {
  default = "10.100.0.0/16"
}

variable "aws_subnet_cidr" {
  default = "10.100.1.0/24"
}

variable "azure_vnet_cidr" {
  default = "10.200.0.0/16"
}

variable "azure_subnet_cidr" {
  default = "10.200.1.0/24"
}

variable "gcp_subnet_cidr" {
  default = "10.210.1.0/24"
}
```

Teaching note for students: point out that the three ranges (10.100.0.0/16, 10.200.0.0/16, 10.210.0.0/16 conceptually) were chosen deliberately to **never overlap** — this is the single most important planning step before any real cross-cloud VPN or peering connection is attempted, since overlapping ranges make routing ambiguous.

Step 4 — `aws.tf`

```

resource "aws_vpc" "main" {
  cidr_block      = var.aws_vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "mc-lab3-aws-vpc" }
}

resource "aws_subnet" "main" {
  vpc_id      = aws_vpc.main.id
  cidr_block = var.aws_subnet_cidr
  tags = { Name = "mc-lab3-aws-subnet" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}

resource "aws_route_table_association" "assoc" {
  subnet_id      = aws_subnet.main.id
  route_table_id = aws_route_table.public.id
}

```

Step 5 — azure.tf

```

resource "azurerm_resource_group" "rg" {
  name      = "rg-mc-lab3"
  location = "East US"
}

resource "azurerm_virtual_network" "vnet" {
  name            = "mc-lab3-vnet"
  address_space   = [var.azure_vnet_cidr]
  location        = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
  name            = "mc-lab3-subnet"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes   = [var.azure_subnet_cidr]
}

```

Step 6 — gcp.tf

```

resource "google_compute_network" "vpc" {
  name            = "mc-lab3-vpc"
  auto_create_subnetworks = false
}

```

```

}

resource "google_compute_subnetwork" "subnet" {
  name          = "mc-lab3-subnet"
  ip_cidr_range = var.gcp_subnet_cidr
  region        = "us-central1"
  network       = google_compute_network.vpc.id
}

```

Step 7 — outputs.tf — the consolidated network map

```

output "multicloud_network_map" {
  description = "Consolidated view of all three cloud networks for architecture review"
  value = {
    aws = {
      vpc_id      = aws_vpc.main.id
      vpc_cidr    = aws_vpc.main.cidr_block
      subnet_cidr = aws_subnet.main.cidr_block
    }
    azure = {
      vnet_id      = azurerm_virtual_network.vnet.id
      vnet_cidr    = azurerm_virtual_network.vnet.address_space[0]
      subnet_cidr = azurerm_subnet.subnet.address_prefixes[0]
    }
    gcp = {
      network_id = google_compute_network.vpc.id
      subnet_cidr = google_compute_subnetwork.subnet.ip_cidr_range
    }
  }
}

```

Step 8 — Run the workflow

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Step 9 — Validate

```

terraform output multicloud_network_map

```

Review the consolidated map as a class and confirm no CIDR ranges overlap — this is exactly the kind of output a cloud architect would attach to a network design document.

5.3.5 Cleanup

```

terraform destroy

```

5.3.6 Challenge / Extension

- Write the `multicloud_network_map` output to a local file using a `local_file` resource (hashicorp/local provider) so it can be committed to a documentation repo.
- Research and diagram (on paper or in a tool like draw.io) what an actual AWS-to-Azure VPN gateway pairing would require, using the CIDRs from this lab.

5.3.7 Troubleshooting Notes

- If `terraform validate` complains about the nested output map, ensure every nested value is a plain string/number — Terraform's `output` blocks handle nested maps fine, but referencing a full resource object (rather than its attributes) can produce unexpected (**known after apply**) noise in the plan.
- Double-check CIDR math with `terraform console` and functions like `cidrsubnet()` if extending this lab with more subnets.

5.4 Lab MC-4: Multi-Cloud Primary/Standby Pattern with Workspaces and Shared Modules (Advanced)

Difficulty: Advanced | **Estimated Time:** 100–120 minutes **Cost note:** this lab runs a small VM in both AWS and Azure simultaneously — modest cost, destroy promptly.

5.4.1 Objective

Simulate a simple **multi-cloud active/standby (DR) pattern**: a “primary” web application on AWS and a “standby” (currently powered down conceptually, but deployed identically) on Azure, built from **one shared local module interface** so the same variables describe both deployments. Use **Terraform workspaces** to separate **primary** and **dr** planning contexts, and use `terraform_remote_state` to let one configuration read outputs from another.

5.4.2 Prerequisites

- Completion of Labs MC-1 through MC-3
- Completion of the Advanced labs in AWS and Azure (understanding of modules and remote backends)

5.4.3 Architecture Overview

- **Stack A (AWS - primary)**: VPC, subnet, security group, 1 EC2 instance, state stored in S3 backend, tagged `role = "primary"`
- **Stack B (Azure - standby)**: VNet, subnet, NSG, 1 Linux VM, state stored in Azure Storage backend, tagged `role = "standby"`
- **Stack C (DR dashboard)**: a small root module with **no cloud resources of its own** that uses `terraform_remote_state` data sources to read both Stack A’s and Stack B’s outputs, and prints a consolidated failover status report — demonstrating how organizations build cross-cloud visibility on top of independently-managed Terraform states

5.4.4 Step-by-Step Instructions

Step 1 — Bootstrap remote state backends

Reuse (or recreate, if destroyed) the AWS S3 backend from Lab AWS-3 and the Azure Storage backend from Lab AZ-3. Confirm you know: - Your AWS state bucket name and DynamoDB lock table name - Your Azure state storage account name and container name

Step 2 — Stack A: AWS “primary” — project setup

```
mkdir -p terraform-multicloud-lab4/stack-a-aws
cd terraform-multicloud-lab4/stack-a-aws
```

backend.tf:

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket      = "tf-state-lab3-XXXXXXX" # reuse from Lab AWS-3
  key         = "mc-lab4/aws-primary.tfstate"
  region      = "us-east-1"
  dynamodb_table = "tf-lock-lab3"
  encrypt     = true
}
```

```

    }
  }

  provider "aws" {
    region = "us-east-1"
  }

main.tf:

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name     = "vpc-id"
    values   = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

resource "aws_security_group" "primary_sg" {
  name     = "mc-lab4-primary-sg"
  vpc_id   = data.aws_vpc.default.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "primary" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"
  subnet_id     = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.primary_sg.id]

  user_data = <<-EOF
    #!/bin/bash

```

```

yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>PRIMARY (AWS) - Active</h1>" > /var/www/html/index.html
EOF

```

```

tags = { Name = "mc-lab4-primary", role = "primary" }
}

```

```

output "primary_ip" {
  value = aws_instance.primary.public_ip
}

```

```

output "role" {
  value = "primary"
}

```

```

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Step 3 — Stack B: Azure “standby” — project setup

```

cd ../../
mkdir -p terraform-multicloud-lab4/stack-b-azure
cd terraform-multicloud-lab4/stack-b-azure

```

backend.tf:

```

terraform {
  required_version = ">= 1.6.0"
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.90"
    }
  }
  backend "azurerm" {
    resource_group_name = "rg-tfstate-lab3"           # reuse from Lab AZ-3
    storage_account_name = "tfstatelab3XXXXXXXXXX"
    container_name      = "tfstate"
    key                  = "mc-lab4-azure-standby.tfstate"
  }
}

provider "azurerm" {
  features {}
}

```

main.tf:

```

resource "azurerm_resource_group" "rg" {
  name     = "rg-mc-lab4-standby"
  location = "East US"
}

```

```

resource "azurerm_virtual_network" "vnet" {
  name                = "mc-lab4-vnet"
  address_space       = ["10.240.0.0/16"]
  location             = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
  name                = "mc-lab4-subnet"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.240.1.0/24"]
}

resource "azurerm_public_ip" "pip" {
  name                = "mc-lab4-standby-pip"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Static"
  sku                 = "Standard"
}

resource "azurerm_network_security_group" "nsg" {
  name                = "mc-lab4-standby-nsg"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  security_rule {
    name                = "Allow-HTTP"
    priority             = 100
    direction           = "Inbound"
    access              = "Allow"
    protocol             = "Tcp"
    source_port_range    = "*"
    destination_port_range = "80"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}

resource "azurerm_network_interface" "nic" {
  name                = "mc-lab4-standby-nic"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                = "internal"
    subnet_id           = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = azurerm_public_ip.pip.id
  }
}

resource "azurerm_network_interface_security_group_association" "nic_nsg" {

```

```

    network_interface_id      = azurerm_network_interface.nic.id
    network_security_group_id = azurerm_network_security_group.nsg.id
}

resource "azurerm_linux_virtual_machine" "standby" {
  name                        = "mc-lab4-standby-vm"
  resource_group_name        = azurerm_resource_group.rg.name
  location                   = azurerm_resource_group.rg.location
  size                       = "Standard_B1s"
  admin_username             = "azureuser"
  network_interface_ids      = [azurerm_network_interface.nic.id]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("~/ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  custom_data = base64encode(<<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y apache2
    echo "<h1>STANDBY (Azure) - Passive</h1>" > /var/www/html/index.html
    systemctl enable apache2
    systemctl start apache2
    EOF
  )

  tags = { role = "standby" }
}

output "standby_ip" {
  value = azurerm_public_ip.pip.ip_address
}

output "role" {
  value = "standby"
}

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

```

Step 4 — Stack C: DR Dashboard (cross-cloud remote state reader)

```
cd ../../  
mkdir -p terraform-multicloud-lab4/stack-c-dashboard  
cd terraform-multicloud-lab4/stack-c-dashboard
```

main.tf:

```
terraform {  
  required_version = ">= 1.6.0"  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}  
  
data "terraform_remote_state" "aws_primary" {  
  backend = "s3"  
  config = {  
    bucket = "tf-state-lab3-XXXXXXX"  
    key     = "mc-lab4/aws-primary.tfstate"  
    region = "us-east-1"  
  }  
}  
  
data "terraform_remote_state" "azure_standby" {  
  backend = "azurerm"  
  config = {  
    resource_group_name = "rg-tfstate-lab3"  
    storage_account_name = "tfstatelab3XXXXXXXX"  
    container_name       = "tfstate"  
    key                  = "mc-lab4-azure-standby.tfstate"  
  }  
}  
  
output "dr_status_report" {  
  value = {  
    primary_cloud = "AWS"  
    primary_ip    = data.terraform_remote_state.aws_primary.outputs.primary_ip  
    primary_role  = data.terraform_remote_state.aws_primary.outputs.role  
    standby_cloud = "Azure"  
    standby_ip    = data.terraform_remote_state.azure_standby.outputs.standby_ip  
    standby_role  = data.terraform_remote_state.azure_standby.outputs.role  
  }  
}
```

Step 5 — Run Stack C

```
terraform init  
terraform validate  
terraform plan
```

```
terraform apply
terraform output dr_status_report
```

Notice this stack **creates no infrastructure of its own** — it only reads the state files from Stacks A and B via `terraform_remote_state` data sources, proving how organizations build read-only dashboards or governance tooling on top of existing Terraform-managed environments across clouds.

Step 6 — Simulate a failover discussion (classroom exercise, no Terraform changes) As a group exercise, discuss: if AWS `us-east-1` had an outage, what would it take to (a) promote the Azure standby’s DNS record to production, (b) verify the standby’s application is running current code, and (c) roll back afterward? This lab intentionally stops short of automating DNS failover — that is a natural “Challenge” extension.

5.4.5 Cleanup

```
cd ../stack-c-dashboard && terraform destroy    # no resources, but clean state entries
cd ../stack-b-azure && terraform destroy
cd ../stack-a-aws && terraform destroy
```

Destroy in this order since Stack C only reads from A and B (no dependency ordering issue), but destroying A/B first is good practice to avoid dangling references while iterating.

5.4.6 Challenge / Extension

- Add Terraform **workspaces** (`terraform workspace new dr-drill`) to Stack A and B so a “drill” copy of each environment can be spun up in parallel without colliding on resource names, then torn down independently.
- Integrate a DNS provider (e.g., Route 53 or Cloudflare) with a weighted or failover routing policy that Stack C’s dashboard also reports on.
- Convert Stacks A and B into calls of one shared module with a `cloud = "aws" | "azure"` variable-driven branch — note real Terraform cannot conditionally select a provider type within one module invocation, so this extension is best explored as a design discussion (e.g., using separate modules per cloud with a common variable *interface*, not a literally shared resource block).

5.4.7 Troubleshooting Notes

- **terraform_remote_state permission errors:** the AWS credentials running Stack C need read access to the S3 state bucket; the Azure credentials need read access to the storage account — these may differ from the credentials used to originally create Stacks A/B.
- If outputs show as `null`, double check the **key** path exactly matches what Stack A/B used in their own **backend** blocks.
- This pattern (read-only cross-stack state reads) is the standard, safe way to share data between independently managed Terraform configurations — avoid trying to **import** another stack’s resources directly, which risks state corruption.

6 Appendix: Full Cleanup Checklist

Use this checklist at the end of the course to confirm no cloud resources remain in any account:

- ☐ AWS: no EC2 instances, ALBs, NAT Gateways, EIPs, RDS instances, or non-empty S3 buckets remain (check `us-east-1` and any other region used)
- ☐ AWS: verify DynamoDB lock tables and Terraform state S3 buckets are deleted (Labs AWS-3, MC-4) if no longer needed
- ☐ Azure: no VMs, Load Balancers, Public IPs, AKS clusters, App Services, or SQL servers remain in any resource group
- ☐ Azure: verify the Terraform state storage account (Labs AZ-3, MC-4) is deleted if no longer needed
- ☐ GCP: no Compute Engine instances, GKE clusters, Cloud SQL instances, or non-empty GCS buckets remain
- ☐ GCP: verify the Terraform state GCS bucket (Lab GCP-3) is deleted if no longer needed
- ☐ All local `kubeconfig` files and SSH keys generated for the labs are removed from shared/lab machines
- ☐ All `TF_VAR_*` environment variables containing passwords are unset (`unset TF_VAR_db_password`, etc.)

Instructors: consider setting up billing alerts on each student's cloud account before the course begins, as an extra safety net against forgotten resources.